LUCID

Clarity Through the Chaos.

Project Technical Handbook

Dual-Mode AI Cybersecurity Analysis Platform

VERSION	3.2 Final
PUBLISHED	September 6, 2026
PRODUCTION REVISION	lucid-00008-4bx
RELEASE COMMIT	02d2e1575fcba2d596c704f99a95f2fcb9a53466

Live Application • GitHub Repository

Contents

Click any section to jump directly to it.

1. Executive Summary

LUCID is a dual-mode AI cybersecurity assistant that translates complex security evidence into useful decisions for two audiences. ShieldMe provides plain-language safety guidance for everyday users, while TIQ (Triage IQ) provides analyst-oriented classification, severity, MITRE ATT&CK-aligned reasoning, observable indicators, and recommended security actions.

The production application uses Node.js and Express, Google Vertex AI with Gemini 2.5 Flash, Firebase Authentication, Cloud Firestore, Sharp for image handling, and Google Cloud Run. Production requests and benchmark runners use the same shared analyzer module, lib/analyzeLucidContent.js, which reduces evaluation-to-production drift.

90.5%	67 / 74	0	74 / 74
Combined strict PASS	Strict PASS	FAIL	ShieldMe verdicts

Release indicator	Validated state
Core regression	23/26 (88.5%)
Generalization	17/18 (94.4%)
Adversarial	27/30 (90.0%)
False positives / false negatives	0 / 0
Malicious -> Safe regressions	0
Production revision	lucid-00008-4bx

Release posture

The 90.5% figure is a controlled project benchmark result, not a universal real-world accuracy guarantee. LUCID remains decision support: important security actions should be verified by a human.

2. Problem Statement and Design Intent

Security alerts are often written for specialists. Everyday users may not know whether a message, login notification, screenshot, authentication prompt, or payment request is benign or dangerous. Security analysts face the opposite problem: they need concise technical classification, severity, evidence, ATT&CK mapping, and action guidance rather than a generic warning.

LUCID addresses both needs through one analysis engine with mode-specific prompts and output schemas. The objective is not to replace human judgment, but to reduce interpretation friction and make the same underlying evidence useful at different levels of expertise.

Product principle

One evidence source, two user experiences - simple enough for an everyday user, structured enough for a security analyst.

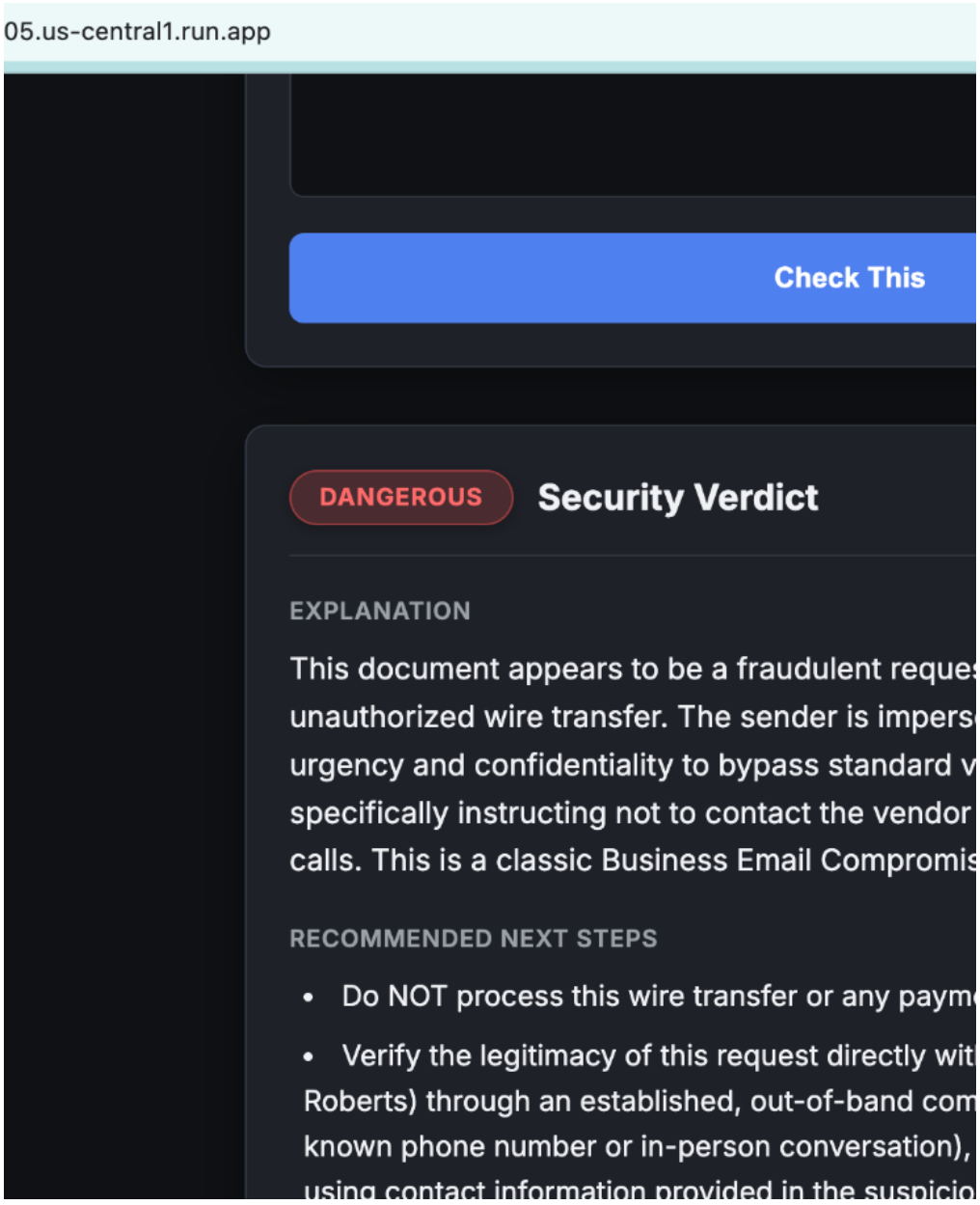
- Persona decoupling: plain-language guidance for non-technical users while preserving structured analyst detail.
- Evidence-first reasoning: model output is constrained by policy and deterministic precedence rules that avoid claiming unobserved compromise.
- Execution parity: production and evaluation run through the same analyzer function.
- Threat-focused persistence: Safe ShieldMe checks are returned to the user but intentionally excluded from My Checks history.

3. User Modes: ShieldMe and TIQ (Triage IQ)

3.1 ShieldMe

ShieldMe uses everyday mode. It returns a Safe, Suspicious, or Dangerous verdict, a plain-English explanation, and actionable next steps. The interface intentionally hides analyst-oriented SecOps complexity.

- Accepts text, image, or combined evidence.
- Prioritizes understandable guidance over security jargon.
- Safe checks are not persisted to My Checks; Suspicious and Dangerous checks may be stored as incidents.



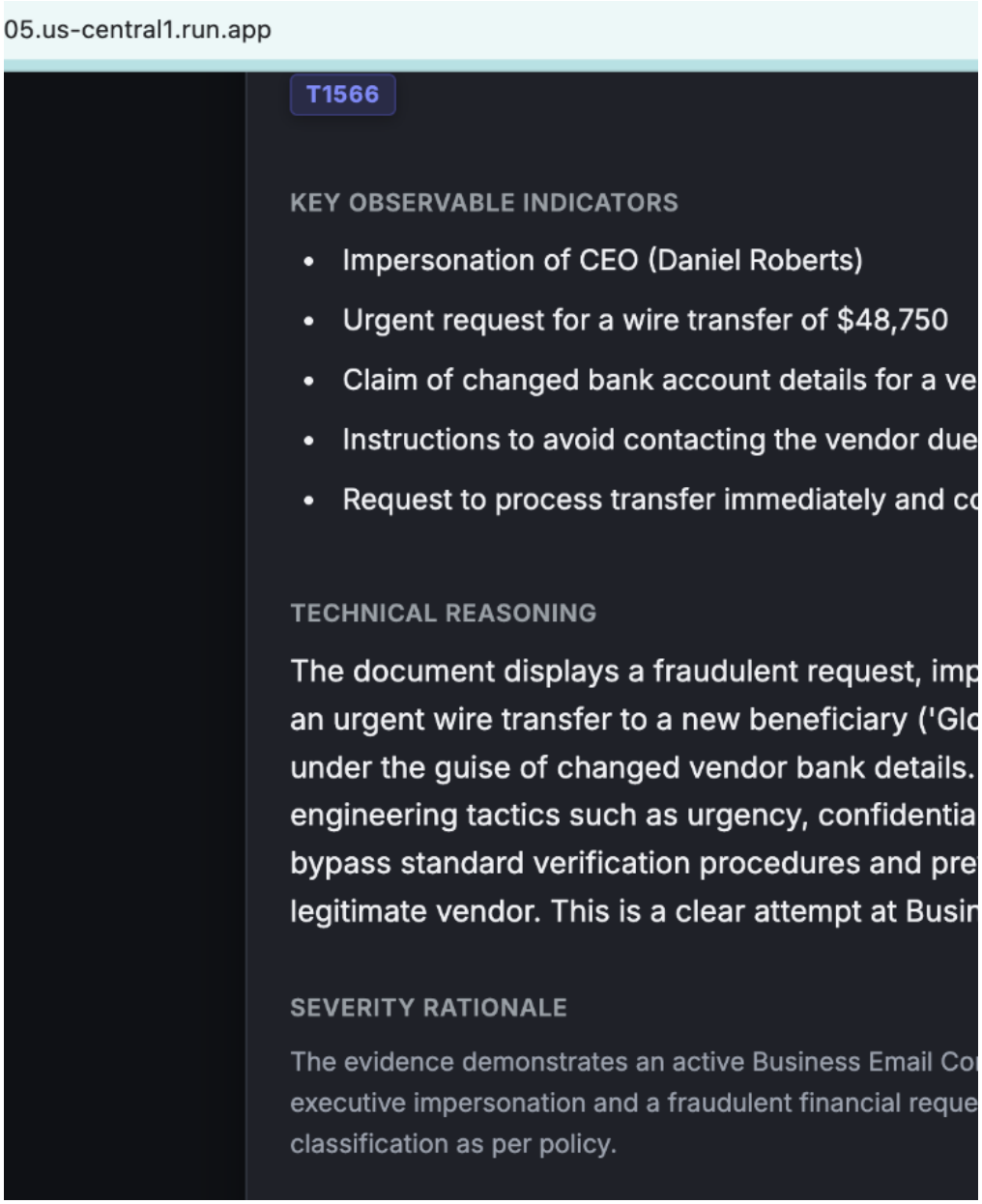
Product view - ShieldMe converts a suspicious payment request into a plain-language verdict and next steps.

3.2 TIQ (Triage IQ)

TIQ uses analyst mode. It returns technical classification, Low/Medium/High/Critical severity, evidence-supported MITRE ATT&CK techniques, indicators, technical reasoning, severity rationale, attack progression when appropriate, and recommended analyst actions.

- Severity is evidence-calibrated rather than keyword-driven.

- Canonical classifications are used instead of benchmark IDs or expected-answer lookup logic.
- ATT&CK mappings are emitted only when supported by observable evidence.



Product view - TIQ (Triage IQ) exposes classification, severity, ATT&CK context, indicators, and response guidance.

4. System Architecture

The production architecture separates browser presentation, authenticated API handling, shared analysis, model inference, and persistence. Tenant identity is derived server-side from the verified Firebase token rather than trusted from client input.

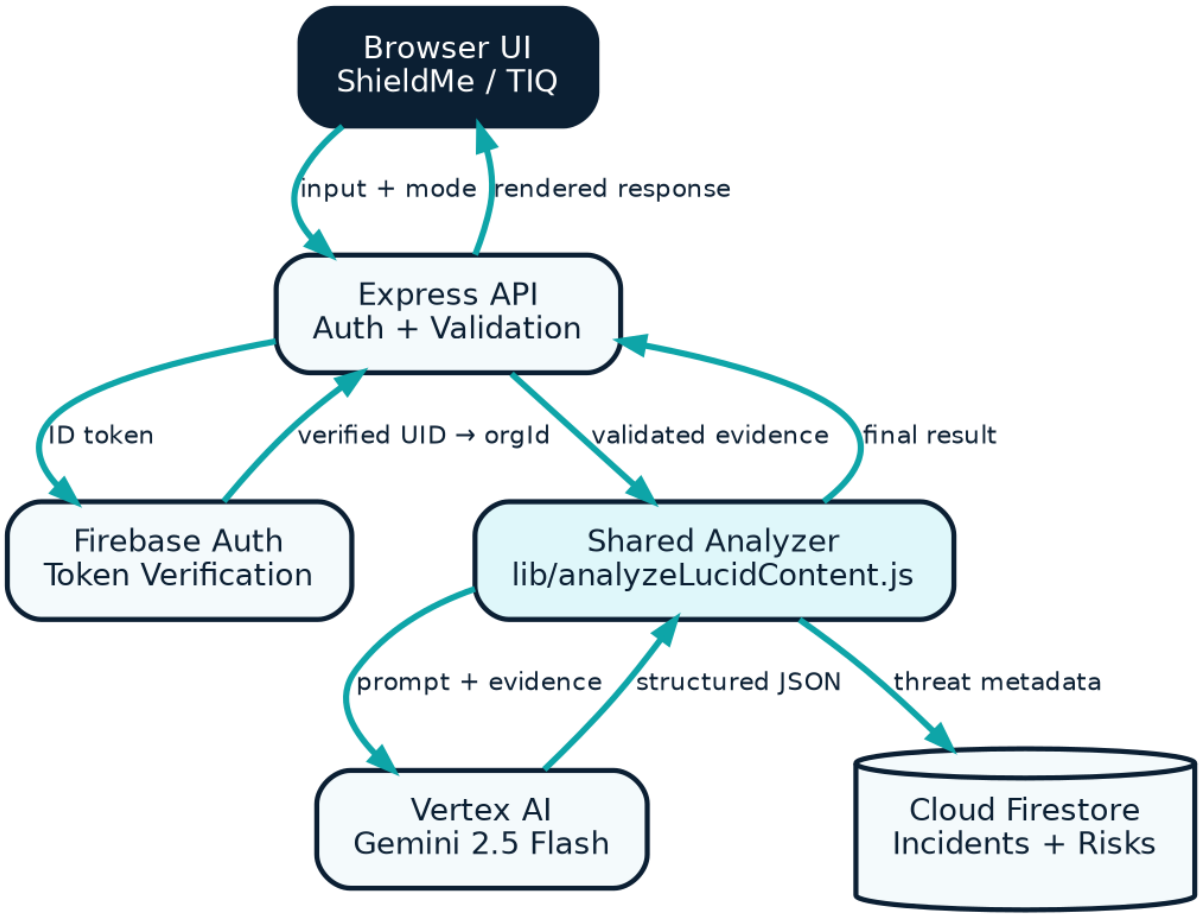


Figure 1 - Current production architecture. Dark navy denotes the current application boundary; cyan indicates authenticated or structured data flow.

- Browser UI: public/index.html and public/script.js collect text/image evidence and mode selection.
- Express API: validates input, applies rate limiting, verifies Firebase ID tokens, and derives orgId from the decoded UID.
- Shared analyzer: lib/analyzeLucidContent.js builds prompts, preprocesses images, invokes Vertex AI, validates JSON, and applies evidence consistency.
- Firestore: stores incident/risk metadata using organization scoping where persistence policy allows.
- Vertex AI: Gemini 2.5 Flash is called in us-central1 with temperature 0 and structured JSON output.

5. Request/Processing Flow

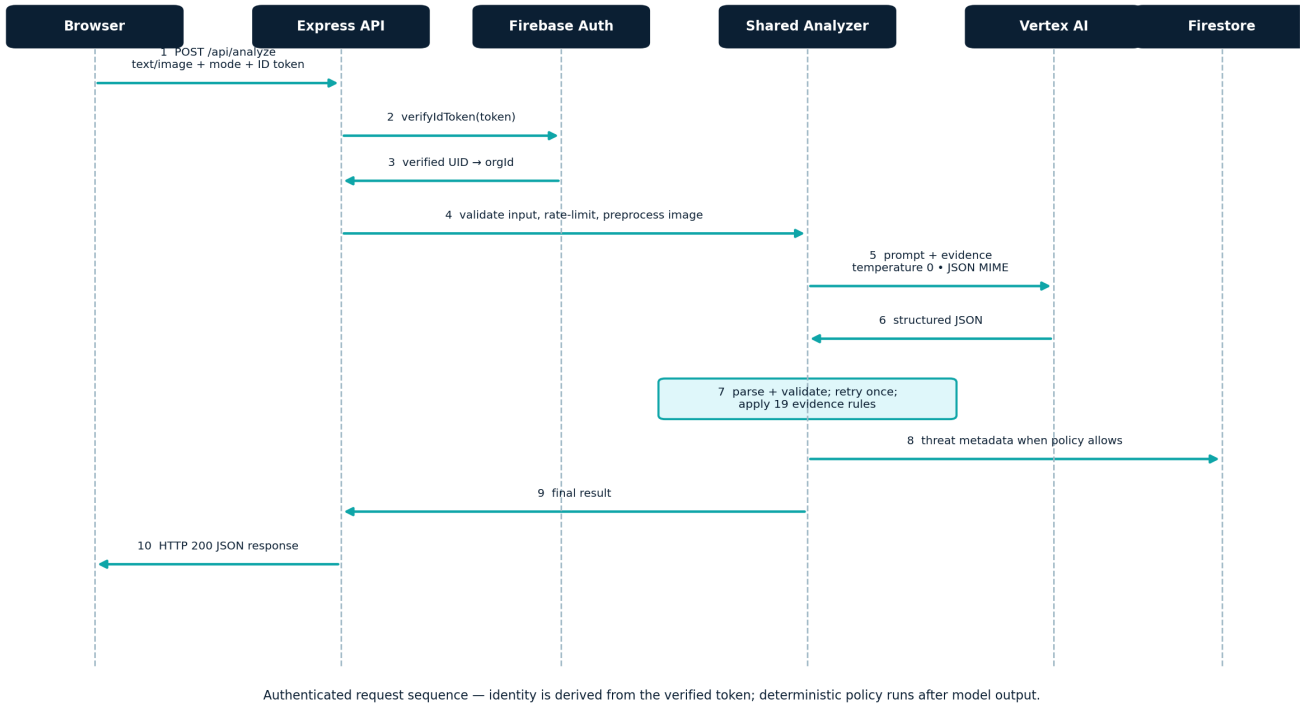


Figure 2 - Authenticated analysis request sequence.

5.1 Detailed Hop Mapping

1. Client authentication (public/script.js): fetches a fresh Firebase ID token and attaches Authorization: Bearer <token>.
2. Rate limiting (server.js): express-rate-limit restricts each client IP to 100 requests per 15-minute window.
3. Token verification and tenant scoping (server.js): Firebase Admin verifies the token; decodedToken.uid becomes req.orgId.
4. Input validation: the API requires text and/or imageBase64 and accepts only everyday or analyst mode.
5. Image pipeline (lib/analyzeLucidContent.js): validates the decoded image against the 8MB ceiling; Sharp strips metadata, resizes to max 1600x1600, and re-encodes WebP at quality 80.
6. Prompt assembly and inference: the analyzer injects the current system reference time, shared policy, mode-specific schema, and evidence before invoking Gemini 2.5 Flash.
7. JSON parse/validation: the response is cleaned, parsed, checked for required fields, and retried once if malformed.
8. Deterministic evidence precedence: applyEvidenceConsistency evaluates 19 rules after model output.
9. Incident persistence: threat-relevant metadata is written asynchronously to Firestore with orgId scoping.
10. Sanitized presentation: the client escapes dynamic HTML before rendering the final result.

Two review-critical controls

Tenant scope comes from a verified token, never from a client-provided orgId. Deterministic policy runs after model inference, so the model proposes and evidence policy can correct or constrain the result.

6. Core Logic and AI Pipeline

6.1 Single Source of Truth

The core design decision is to keep production prompt construction, response parsing, validation, and evidence post-processing in lib/analyzeLucidContent.js. server.js delegates analysis to this module, and benchmark runners import the same analyzer.

Anti-overfitting control

Production analysis code contains no TC-, GEN-, or ADV- benchmark identifiers and no case-specific expected-answer lookup logic.

6.2 Model Configuration

Parameter	Production choice	Purpose
Model	Gemini 2.5 Flash via Vertex AI	Fast multimodal security analysis
Temperature	0	Reduce response randomness during controlled evaluation
Response MIME type	application/json	Encourage machine-parseable structured output
Modes	everyday / analyst	Different output depth from the same evidence base
Region	us-central1	Current Vertex AI deployment location

6.3 Pipeline Flow

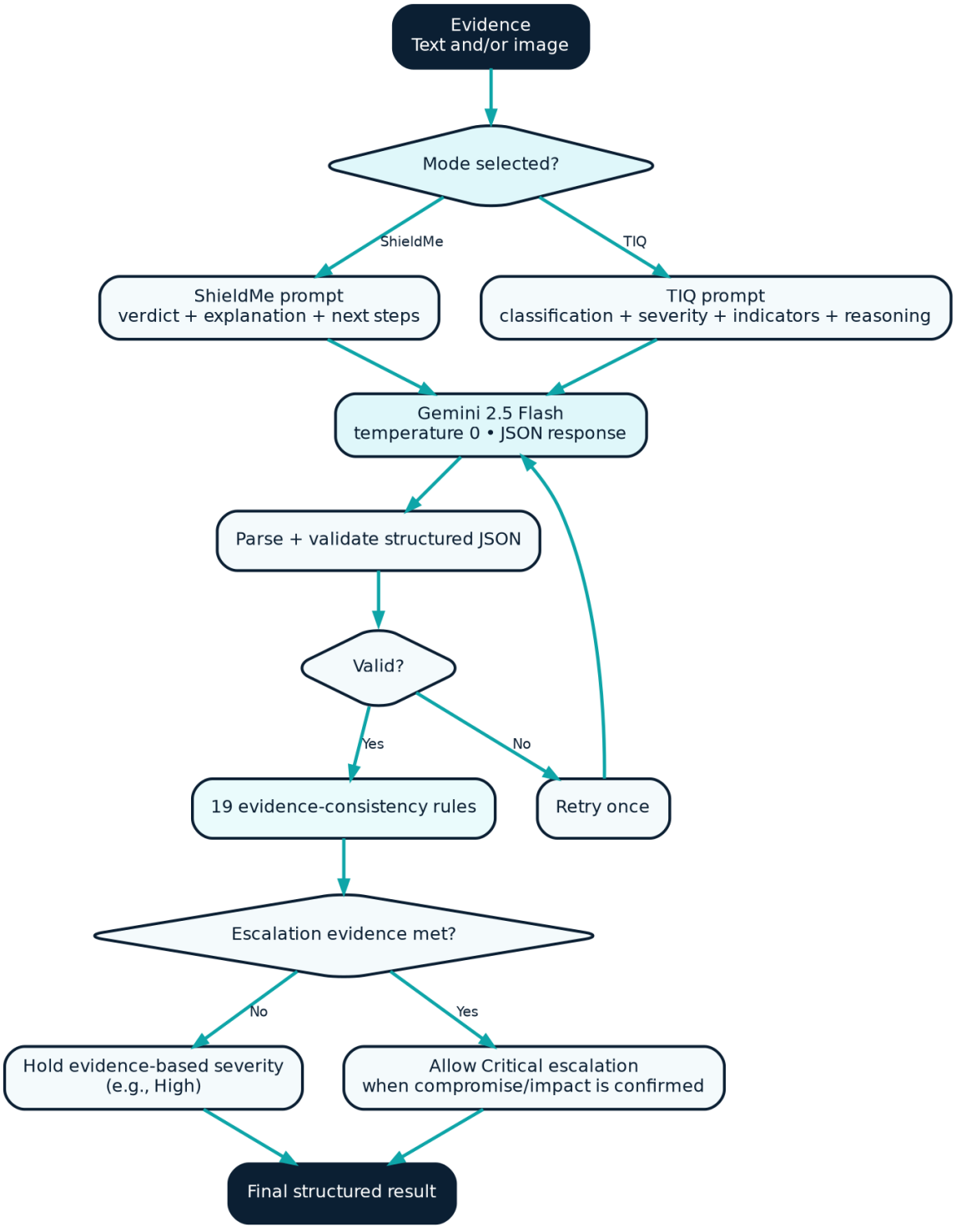


Figure 3 - Shared AI pipeline. Model inference is followed by parse/validation and deterministic evidence-consistency enforcement.

6.4 Structured Response Handling

ShieldMe and TIQ use different JSON contracts. The analyzer validates required fields and retries once when the model response cannot be parsed or does not satisfy the expected structure. Everyday mode also enforces the next-steps contract before the final response is returned.

ShieldMe fields	TIQ fields
verdict, explanation, next_steps[]	verdict, classification, severity, mitre_attack[], key_indicators[], technical_reasoning, attack_chain, why_severity, recommended_action, soc_actions[], confidence, unknowns

7. Evidence-First Security Reasoning - Full Rule Set

LUCID combines prompt-level security policy with deterministic post-processing. The purpose is not to replace the model with a keyword classifier, but to protect high-impact decisions from unsupported escalation or unsafe de-escalation when decisive evidence is explicitly present.

7.1 Summary Principles

- Do not treat administrative execution flags, explicit script paths, or privileged context as automatically benign.
- Do not trust sender domain or aligned URL alone for password-reset or security emails.
- Apply malicious-to-Safe safety gates separately across Core, Generalization, Adversarial, and combined evaluation.
- Do not force unconfirmed RCE attempts or BEC requests to Critical without evidence of successful execution, transfer, or comparable impact.
- MITRE ATT&CK mapping is evidence-based only.
- Multi-Stage Intrusion is reserved for genuinely correlated multi-stage chains.
- Temporal inconsistency alone is contextual evidence, not proof of compromise; Impossible Travel requires distinct geographic authentication events with implausible timing.

7.2 Deterministic Override Pipeline

#	Rule	Verified source range	Behavior
1	Account Enumeration	L312-L334	Password-reset or username-discovery probing can be classified as account enumeration when evidence supports it.
2	MFA Fatigue / Push Bombing	L336-L354	Repeated unsolicited MFA prompts with attack context map to MFA fatigue and T1621.
3	OAuth / Illicit Consent Grant	L356-L379	Untrusted OAuth consent requesting dangerous scopes is treated as illicit consent behavior.
4	Phishing / Credential Harvesting	L381-L453	Deceptive login or financial lures with credential-harvesting evidence can be escalated; aligned sender/domain alone is insufficient to mark Safe.
5	Business Email Compromise	L455-L493	Executive/vendor impersonation plus payment or sensitive-business action can be BEC; urgency or amount alone does not force Critical.
6	Ransomware Activity	L495-L519	Mass encryption and destructive recovery inhibition support ransomware classification and Critical impact.
7	Web Shell Upload / RCE	L521-L546	Web shell behavior with confirmed command execution supports RCE; unconfirmed exploit attempts are not automatically Critical.
8	Command Injection	L548-L573	OS command injection with execution evidence supports Command Injection and Critical severity.
9	Unauthorized Domain Admin Creation	L575-L603	Unauthorized account creation followed by privileged group membership supports Critical identity compromise.
10	Malicious Macro Execution	L605-L638	Office macro execution that launches a downloader or malicious child process supports macro execution classification.
11	Malware Staging & Persistence	L640-L662	Dropper/download plus persistence evidence supports malware staging/persistence.
12	Privilege Escalation	L664-L682	Explicit transition from unprivileged context to root/SYSTEM supports privilege escalation.
13	Multi-Stage Intrusion	L684-L719	Requires a correlated chain spanning at least three distinct malicious stages; not used as a generic umbrella label.

#	Rule	Verified source range	Behavior
14	Lateral Movement / Domain Controller	L721-L761	Requires a compromised or malicious source, a remote-execution mechanism, and a remote target. Domain-controller targeting yields Critical; otherwise High unless already Critical. Default mappings include T1021 and, for DC/WMI evidence, T1047.
15	Confirmed Data Exfiltration	L765-L799	Requires sensitive data, actual transfer/exfiltration, and an external or unauthorized destination. Major compromise is Critical; otherwise High. Default mapping T1048.
16	Benign Administrative PowerShell	L802-L825	Scheduled or routine administrative scripts with benign context are prevented from being escalated merely due to PowerShell flags or script paths.
17	Malicious Obfuscated PowerShell	L827-L848	Encoded or hidden PowerShell with malicious execution context supports PowerShell Execution / Obfuscated PowerShell and T1059.001.
18	Routine Enterprise Authentication	L850-L893	Legitimate SSO/SAML login context is protected from unsupported phishing, Impossible Travel, or T1078 claims.
19	Impossible Travel Anomaly	L895-L922	Requires at least two geographically distinct authentication events for the same user with implausible travel timing/velocity; one login, timestamp discrepancy, or MFA event is insufficient.

8. Multimodal Analysis

8.1 Image Processing

When imageBase64 is supplied, the analyzer validates the decoded buffer against an 8MB ceiling, uses Sharp to remove metadata, resizes the image to a maximum 1600x1600 boundary, and re-encodes it to WebP quality 80 before model inference. The cleaned image and text evidence, when both are present, are evaluated together.

8.2 Temporal Hardening

Temporal hardening was introduced after a benign Microsoft sign-in screenshot exposed a date/time hallucination risk. The policy now treats timestamp discrepancies as context rather than standalone proof, uses the current UTC system reference time, allows for screenshot-local timezones and date-boundary ambiguity, and requires two distinct geographic authentication events before Impossible Travel can be asserted.

Temporal evidence rule

A single login, a one-calendar-day discrepancy around timezone boundaries, or MFA presence alone is insufficient for Impossible Travel or Valid Accounts (T1078).

9. Authentication, Authorization, and Data Isolation

Figure 4 - Authentication and tenant-isolation flow.

Protected routes require a Firebase Bearer ID token. Firebase Admin verifies the token and the decoded UID is used as orgId. Firestore queries and mutations are scoped to that orgId; incident and risk updates use allow-listed fields and ownership checks.

The browser client configures Firebase Auth with SESSION persistence before Google Sign-In. Authentication survives page refreshes within the active LUCID session, while persistent local sign-in across newly opened LUCID sessions is intentionally disabled. In the verified production browser flow, closing the LUCID tab requires Google Sign-In when LUCID is opened again.

- Client-supplied tenant identity is not trusted.
- Cross-tenant access is blocked by server-side scope and Firestore rules.
- Risk PATCH permits description, category, priority, and status.
- Incident PATCH is restricted to status and notes.

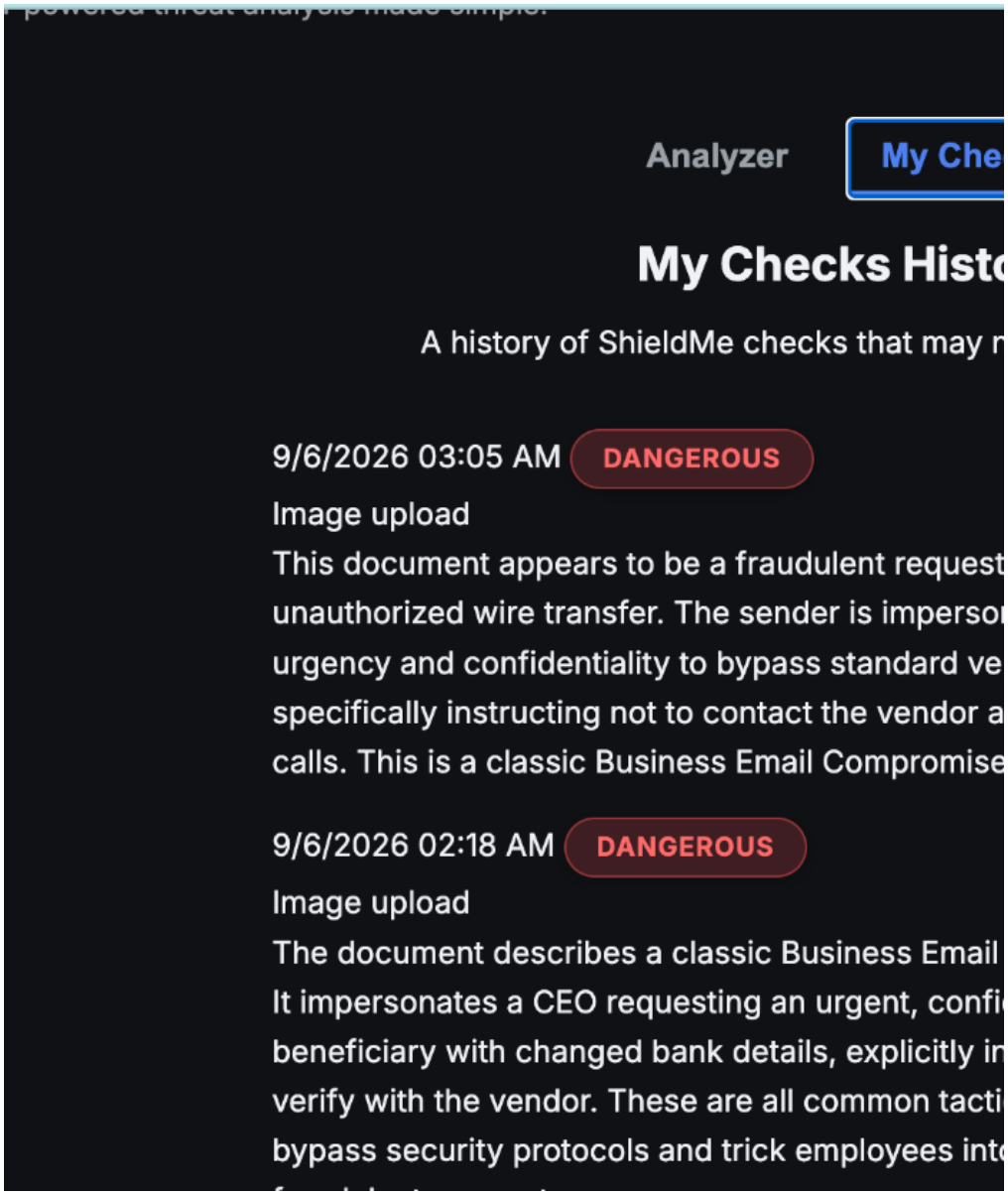
10. Firestore Data Model and My Checks Behavior

Firestore stores incident and risk metadata rather than acting as a raw evidence archive. Incident records are tenant-scoped with orgId. Composite indexes support common orgId + status + severity query patterns.

Collection / behavior	Purpose
incidents	Threat-relevant analysis metadata and status/notes workflow
risks	Risk Register entries with controlled editable fields
My Checks	Threat-focused ShieldMe history; Safe checks intentionally omitted
Composite indexes	Support scoped status/severity filtering

10.1 Threat-Focused My Checks History

Safe ShieldMe checks are intentionally not persisted. Suspicious or Dangerous checks may be stored and surfaced in My Checks, whose user-facing description is: "A history of ShieldMe checks that may need your attention." This is a product choice, not an incomplete audit trail.



Product view - My Checks intentionally focuses on ShieldMe checks that may need attention.

11. Frontend and API Security Controls

Control	Current behavior
Authentication	Firebase ID token required for protected APIs
Rate limiting	100 requests per 15 minutes per IP
Input validation	Mode and evidence presence validated server-side
Image handling	8MB cap; Sharp metadata stripping; max 1600x1600 WebP
Dynamic HTML	Rendered values are passed through escapeHtml before DOM insertion
CSV export	Potential formula-injection prefixes = + - @ are neutralized
CORS	FRONTEND_ORIGIN or localhost development fallback
Mutation safety	Allow-listed PATCH fields plus org ownership checks

Security scope

These controls reduce common application risks but do not constitute a formal penetration test, independent security audit, or compliance certification.

12. Benchmark and Evaluation Methodology

LUCID uses three automated suites covering controlled core regression cases, unseen generalization cases, and adversarial/unseen cases. Each runner exercises the same shared production analyzer used by the deployed API.

Suite	Runner / evaluator	Cases	PASS	PARTIAL	FAIL
Core Regression	run_accuracy_suite.js / evaluate_results.js	26	23	3	0
Generalization	run_generalization_suite.js / evaluate_generalization.js	18	17	1	0
Adversarial	run_adversarial_suite.js / evaluate_adversarial.js	30	27	3	0
Combined	All three suites	74	67	7	0

The final configuration cleanup was followed by evaluator execution across all 74 cases. Results matched the pre-change release metrics exactly: 67 strict PASS, 7 PARTIAL, 0 FAIL, 0 false positives, and 0 false negatives.

12.1 Evaluation Dimensions

- ShieldMe verdict correctness.
- TIQ classification correctness and canonical taxonomy alignment.
- Severity calibration.
- MITRE ATT&CK mapping quality when applicable.

- Explanation/reasoning quality and actionable remediation.
- Safety gates for false positives, false negatives, and malicious-to-Safe regressions.

12.2 PASS, PARTIAL, and FAIL

Strict PASS requires the evaluated criteria to match expected behavior. PARTIAL is reported separately when the threat is substantially understood but one strict dimension, such as severity or exact taxonomy, differs. FAIL is reserved for a material miss. PARTIAL results are not counted as strict PASS.

13. Final Release Results

90.5%	88.5%	94.4%	90.0%
Combined	Core	Generalization	Adversarial

Suite	PASS	PARTIAL	FAIL	Strict PASS rate
Core	23/26	3	0	88.5%
Generalization	17/18	1	0	94.4%
Adversarial	27/30	3	0	90.0%
Combined	67/74	7	0	90.5%

- 0 FAIL across 74 controlled benchmark cases.
- ShieldMe verdicts: 74/74.
- False positives / false negatives: 0 / 0.
- Malicious-to-Safe regressions: 0.

Interpretation

The 90.5% result exceeds the project target of 80% on this controlled suite. It should be reported as suite-specific validation, not as a universal accuracy claim for every real-world threat.

13.1 Why Some PARTIAL Results Were Intentionally Preserved

Several remaining PARTIAL results reflect deliberate evidence-first choices rather than unfixed safety defects. An active BEC request without confirmed transfer remains High rather than being forced to Critical, and an exploit payload without confirmed execution is not automatically escalated to Critical. Altering those rules only to raise a benchmark score would increase overfitting risk.

14. Multimodal Smoke Validation

Scenario	ShieldMe	TIQ (Triage IQ)	Observed result
Benign Microsoft sign-in	SAFE	LOW - Benign / Legitimate	No Impossible Travel, T1078, or phishing
Bank of America phishing	DANGEROUS	HIGH - Financial Phishing / Brand Impersonation	T1566.002
MFA fatigue / push bombing	DANGEROUS	HIGH - MFA Fatigue / Push Bombing	T1621
Obfuscated PowerShell / EDR	DANGEROUS	CRITICAL - PowerShell Execution	Core T1059.001
Confirmed command injection	DANGEROUS	CRITICAL - Command Injection	Execution confirmed by output www-data
Suspicious data transfer	DANGEROUS	CRITICAL - Data Exfiltration	T1567 and T1074.001 included
BEC payment request	DANGEROUS	HIGH - Business Email Compromise	T1566; not forced Critical without confirmed loss

These smoke tests increase confidence in representative multimodal paths but do not prove universal detection capability.

15. Deployment and Release Engineering

The application is containerized and deployed to Google Cloud Run from source. Cloud Build builds the container, Artifact Registry stores build artifacts, and Cloud Run creates immutable service revisions with traffic routing.

Release item	Current validated value
Git branch	main
Final config commit	a3c52c5b0f6e841222c468b6dee80ba1dd4e4790
Commit message	fix(config): require runtime GCP project configuration
Cloud Run revision	lucid-00008-4bx
Traffic	100%
Region	us-central1
Production URL	https://lucid-264271786605.us-central1.run.app/
Repository status	Clean - 0 uncommitted changes

15.1 Runtime Project Configuration Hardening

The final production cleanup removed the hardcoded Vertex AI project-ID fallback from lib/analyzeLucidContent.js. The analyzer now requires GOOGLE_CLOUD_PROJECT or GCLOUD_PROJECT and fails startup explicitly if neither is configured. Cloud Run deployment verified that the runtime supplies GOOGLE_CLOUD_PROJECT and that the service starts successfully.

Verification	Result
Missing env variable test	Expected configuration error thrown
Configured env variable test	Shared analyzer loads successfully

Verification	Result
Evaluator regression	67/74 strict PASS; 7 PARTIAL; 0 FAIL
Production root smoke test	HTTP 200 OK
Unauthenticated /api/analyze	HTTP 401 Unauthorized as expected
Cloud Run traffic	100% to lucid-00008-4bx

15.2 Release Discipline

The release sequence used narrow defect/configuration changes, targeted verification, controlled benchmark evaluation when analysis behavior could be affected, Git commit/push, Cloud Run deployment, and live smoke checks. The application is now frozen unless a real defect is discovered.

16. Operational Limitations and Responsible Use

User-facing disclaimer

AI-generated analysis can make mistakes. Verify important security decisions.

- LUCID is an AI-assisted analysis tool, not an autonomous incident-response authority.
- A screenshot or log excerpt may omit endpoint, identity, network, or cloud context normally used by a SOC analyst.
- The 74-case benchmark is project-generated controlled test data, not an independent certification.
- High-impact containment or remediation should remain subject to human verification and organizational policy.
- Screenshot analysis depends on legible visual evidence.
- LUCID recommends actions but does not autonomously isolate hosts, disable accounts, revoke access, or execute containment.

17. Future Scalability and Enterprise Integration

The current LUCID application is a standalone analysis platform. Its structured API and shared analyzer provide a practical path to future security-operations integrations, but the items below are roadmap concepts and are not currently deployed capabilities.

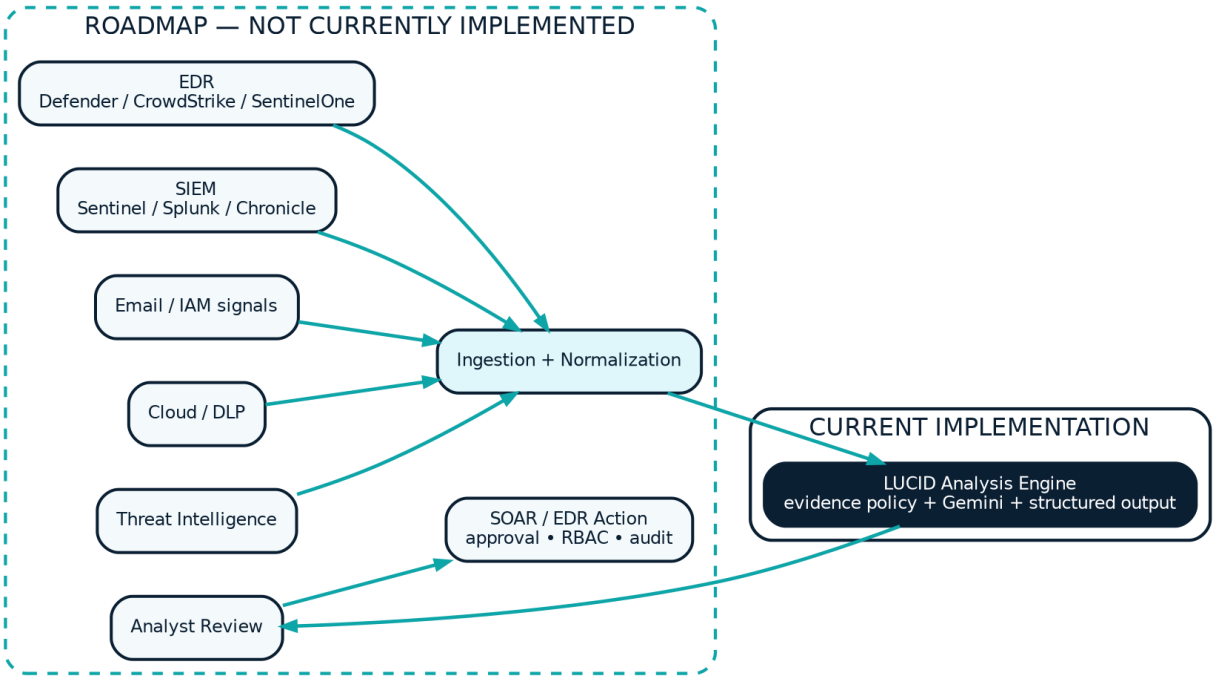


Figure 5 - CURRENT vs ROADMAP integration model. Roadmap adapters and response automation are explicitly not part of the current production release.

Target system	Potential integration point	Prerequisite changes	Feasibility
EDR	Normalize endpoint telemetry into analyzer evidence	Telemetry schema for process trees, hosts, network details	High
SIEM	Forward correlated alerts to an ingestion endpoint	Machine-to-machine auth and vendor normalization	High
Threat intelligence	Enrich evidence before prompt construction	Provenance-aware async enrichment layer	Medium
SOAR	Consume structured output as decision support	RBAC, approvals, audit logs, idempotent action gates	Medium

Future high-volume deployments could use webhooks or Pub/Sub/queue ingestion, normalized evidence storage with explicit retention policy, organization/team RBAC, audit trails, and analyst feedback captured as evaluation data rather than blindly trusted training truth.

18. Known Issues and Gaps

The following items are current limitations identified from the codebase and release process. Resolved release-hardening items are distinguished from remaining gaps so the handbook does not present already-fixed configuration as an open defect.

18.1 Current Code-Level / Operational Gaps

Status	Item	Why it matters
OPEN	Firestore incident logging is asynchronous and non-blocking	A persistence failure should be observable through stronger structured monitoring even though it does not block the user response.
EXPECTED	Firebase web configuration is client-visible	Firebase apiKey/projectId/appId in public JavaScript are normal web-client identifiers, not server secrets. Security depends on correct Auth and Firestore Security Rules.
OPEN	server.js retains a separate Firebase Admin project-ID fallback	The final analyzer cleanup intentionally did not broaden scope to server.js. A future configuration-only cleanup may remove that fallback after separate verification.
OPEN	No formal independent penetration test	Current controls are code-reviewed/project-verified rather than independently audited.
OPEN	No enterprise RBAC / full audit trail	Current tenant model is UID/orgId scoped; enterprise teams would require richer roles, auditability, and secrets/integration governance.
EXPECTED	My Checks is not a complete Safe-check audit log	Threat-focused history is deliberate: Safe ShieldMe checks are not persisted.

18.2 Resolved Release Hardening

Resolved item	Resolution
Vertex AI hardcoded project fallback in shared analyzer	Removed in commit a3c52c5; analyzer now requires GOOGLE_CLOUD_PROJECT or GCLOUD_PROJECT and fails closed when absent.
Rule 14/15 handbook source ranges	Re-verified against live analyzer source: Rule 14 L721-L761; Rule 15 L765-L799. All former verification flags have been cleared.
Temporal / Impossible Travel false escalation risk	Hardened earlier with evidence requirements for distinct geographic events and timezone/date-boundary ambiguity.

18.3 Documentation and Process Notes

- The benchmark is synthetic/project-generated and not independently reviewed.
- Benchmark runners require live GCP ADC for model calls; offline deterministic mocks would improve CI independence.
- TIQ reasoning fields provide transparency, but LUCID does not expose a full raw model request/response audit trail to end users.
- The seven PARTIAL benchmark cases are intentionally reported rather than hidden or forced into PASS.

19. Maintainability and Technical Debt

The production application is frozen unless a real defect is discovered. Post-release work should emphasize platform quality and evidence breadth rather than benchmark chasing.

Area	Future maintenance consideration
Configuration	Consider removing the separate server.js project-ID fallback in a narrowly scoped, independently verified cleanup.

Area	Future maintenance consideration
Evaluation	Add larger independently reviewed datasets and more benign diversity before making broader performance claims.
Observability	Add structured request, latency, Firestore write, model error, and cost metrics.
Enterprise security	Add fine-grained RBAC, audit logging, integration secrets management, and formal security testing.
Data lifecycle	Define explicit retention/deletion policies if enterprise telemetry is stored.
CI testing	Add offline mocks for benchmark plumbing while retaining separate live-model evaluation runs.
SDK lifecycle	Periodically review Google AI SDK support status and migrate only with regression coverage.

20. Reviewer Walkthrough

A concise demonstration can show LUCID's product differentiation without requiring a reviewer to inspect the full codebase.

1. Open the production application and authenticate with Firebase Sign-In.
2. Demonstrate ShieldMe with a suspicious message or screenshot. Point out the Safe/Suspicious/Dangerous verdict, plain-language explanation, and next steps.
3. Toggle to TIQ (Triage IQ) and submit representative security telemetry. Show classification, severity, ATT&CK mapping, indicators, reasoning, and recommended actions.
4. Open My Checks and explain that the history is intentionally threat-focused rather than a complete log of Safe checks.
5. Explain the shared analyzer: production and benchmark runners execute the same lib/analyzeLucidContent.js path.
6. Present the controlled benchmark accurately: 67/74 strict PASS (90.5%), 7 PARTIAL, 0 FAIL; ShieldMe 74/74; 0 FP/FN; 0 malicious-to-Safe regressions.
7. Close with responsible-use boundaries and the CURRENT vs ROADMAP integration diagram.

Appendix A. Technology Stack

Layer	Technology / service
Frontend	HTML, CSS, JavaScript
Backend	Node.js, Express.js
AI	Google Vertex AI - Gemini 2.5 Flash
Authentication	Firebase Authentication
Database	Cloud Firestore
Image processing	Sharp
Hosting	Google Cloud Run
Build	Cloud Build
Container artifacts	Artifact Registry
Evaluation	Node.js benchmark runners and evaluator scripts

Appendix B. Full API Reference

The production server exposes authenticated REST endpoints for analysis, incident/risk workflows, and supporting data. Exact route availability should be treated as code-defined; protected endpoints require a Firebase Bearer ID token.

Endpoint family	Purpose / notes
/api/analyze	Primary ShieldMe/TIQ analysis endpoint; accepts text/image evidence and mode.
/api/incidents	List/read incident data scoped to the authenticated org.
/api/incidents/:id	Controlled incident updates; allow-listed fields such as status/notes.
/api/risks	Risk Register listing/creation workflow.
/api/risks/:id	Controlled risk update/delete with org ownership validation.
/api/* export/reporting routes	Sanitized output; CSV values are guarded against formula injection where applicable.

Authentication contract

All protected API calls use Firebase Bearer ID tokens. The server derives orgId from the verified token rather than accepting tenant scope from request content.

Appendix C. Evidence Log

Evidence item	Verified release state
Shared analyzer	lib/analyzeLucidContent.js is the production/evaluation single source of truth.
Rule 14 source	L721-L761 - Lateral Movement / Domain Controller.
Rule 15 source	L765-L799 - Confirmed Data Exfiltration.
Final controlled benchmark	67/74 strict PASS (90.5%); 7 PARTIAL; 0 FAIL.
Safety metrics	ShieldMe 74/74; 0 FP; 0 FN; 0 malicious-to-Safe regressions.
Final config commit	a3c52c5b0f6e841222c468b6dee80ba1dd4e4790.
Production revision	lucid-00008-4bx, 100% traffic.
Production smoke	Root HTTP 200; unauthenticated /api/analyze HTTP 401.
Repository status after deployment	Clean working tree.

Public documentation should preserve these distinctions: controlled benchmark results are not universal guarantees; Firebase browser configuration is not a secret by itself; and roadmap integrations must not be described as current production capabilities.